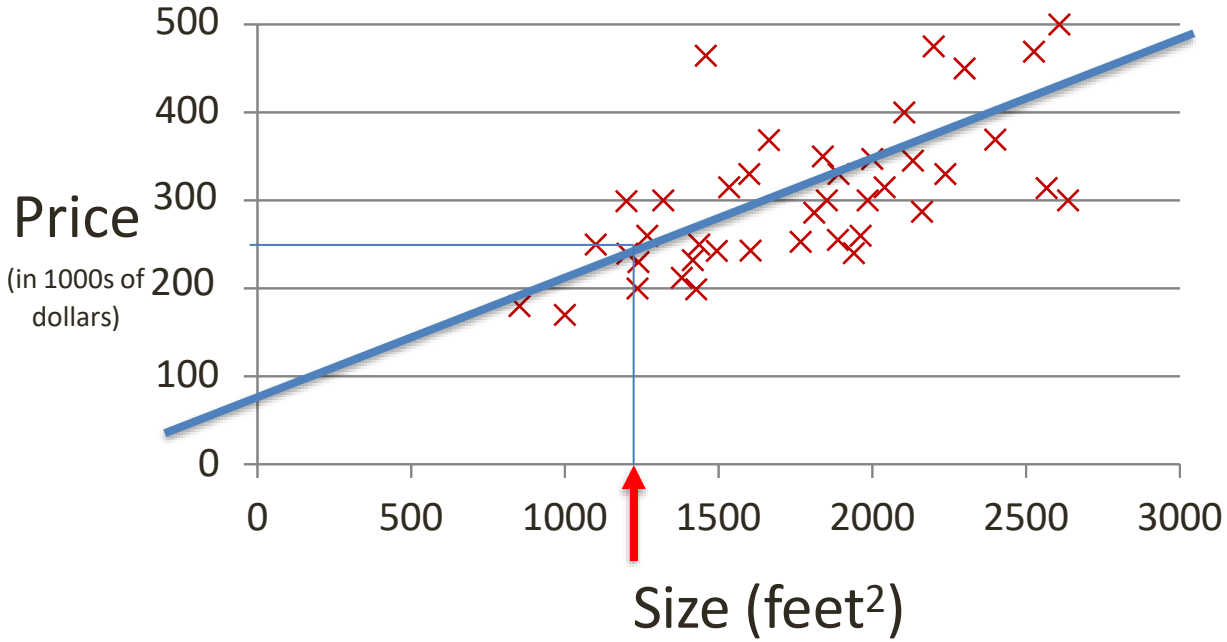
Machine Learning

Instructor: Hafiz Abdul Rehman

Lecture 11: Gradient Descent

Linear Regression with one variable

Size ($feet^2$)	Price \$(×1000)
1500	190
2250	285
2740	420
2318	300
2500	350
1250	180
…	…



- **Notation:**
  - **m** = Number of training samples
  - **x =** Feature
  - **y** = Label
  - **$(x^{(i)}, y^{(i)})$:** the ith sample in the dataset

Linear Regression with one variable

- Linear regression with one variable, univariate linear regression, simple linear regression

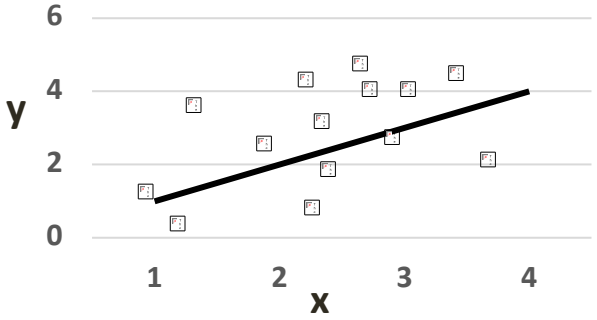
$$\boldsymbol{h_\theta(x) = \theta_0 + \theta_1 x}$$

- **Cost Function**

$$\boldsymbol{J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} (h\theta(xi) - yi)^2}$$

- **Goal**

$$Minimize_{\theta_0,\theta_1} J(\theta_0, \theta_1)$$



Choose θ_0, θ_1 such that $h_\theta(x) \approx y$ for our training examples (x, y).

A Simplified Case

- **Hypothesis**
  - $h\theta(x) = \theta_0 + \theta_1 x$
- **Parameters**

$$\theta_0, \theta_1$$

- **Cost function**

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} (h\theta(xi) - yi)^2$$

- **Goal**

$$Minimize_{\theta_0,\theta_1} J(\theta_0, \theta_1)$$

Assume $\theta_0 = 0$

- **Hypothesis**

$$h\theta(x) = \theta_1 x$$

- **Parameters**

$$\theta_1$$

- **Cost function**

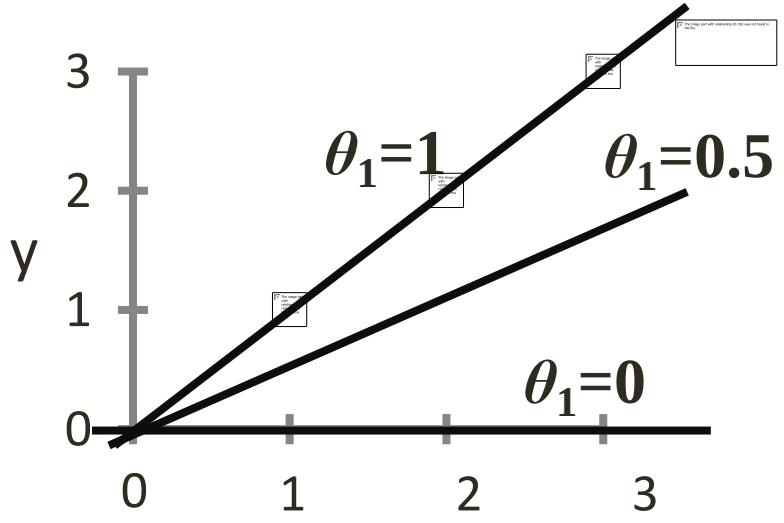
$$J(\theta_1) = \frac{1}{2m} \sum_{i=1}^{m} (h\theta(xi) - yi)^2$$

- **Goal**

$$Minimize_{\theta_0,\theta_1} J(\theta_1)$$

A Simplified Case

- for a fixed θ_0, this is a function of x

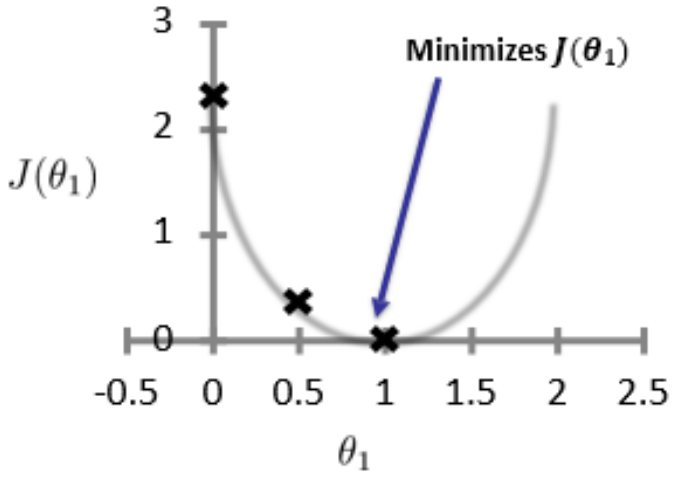


- (function of the parameter θ_1)

$$J(1) = \frac{1}{2(3)}\left((1-1)^2 + (2-2)^2 + (3-3)^2\right) = 0$$

$$J(0.5) = \frac{1}{6}\left((0.5-1)^2 + (1-2)^2 + (1.5-3)^2\right) = 0.58$$

$$J(0) = \frac{1}{6}\left((1)^2 + (2)^2 + (3)^2\right) = 2.3$$



Using both “Knobs”

- **Hypothesis**
  - $\boldsymbol{h\theta(x) = \theta_0 + \theta_1 x}$
- **Parameters**

$$\theta_0, \theta_1$$

- **Cost function**

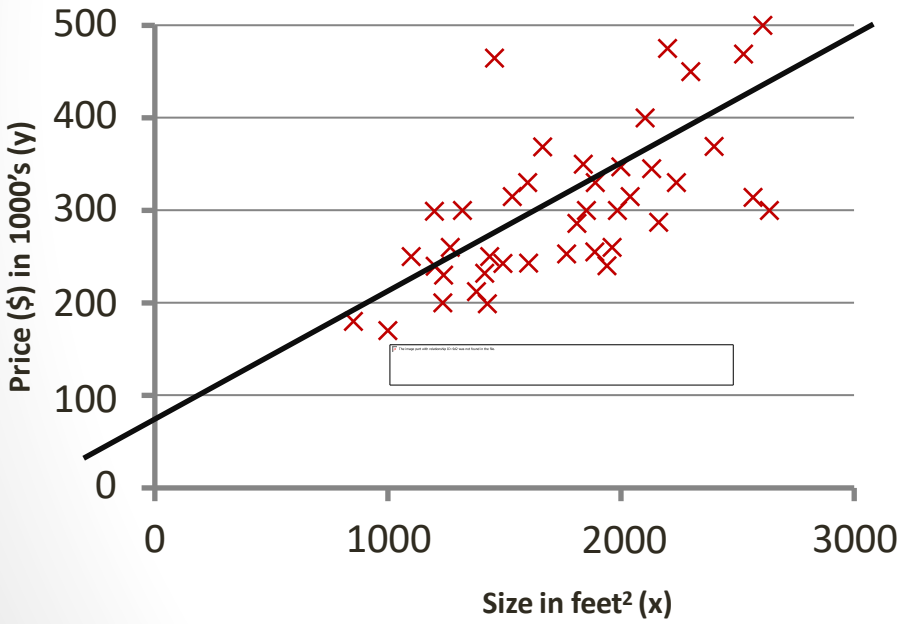
$$\boldsymbol{J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} (h\theta(xi) - yi)^2}$$

- **Goal**

$$Minimize_{\theta_0, \theta_1} J(\theta_0, \theta_1)$$

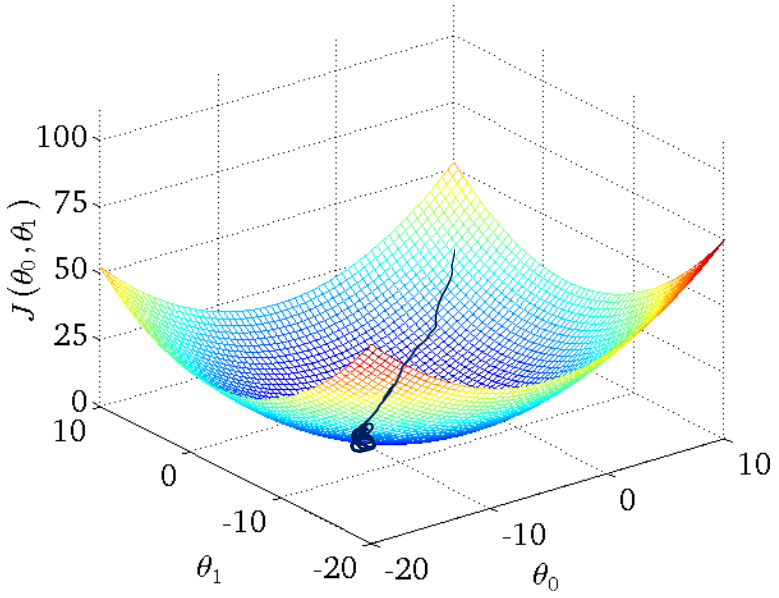
Using both "Knobs"

- for a fixed θ_0, θ_1 this is a function of x
  - $\boldsymbol{h\theta(x) = \theta_0 + \theta_1 x}$



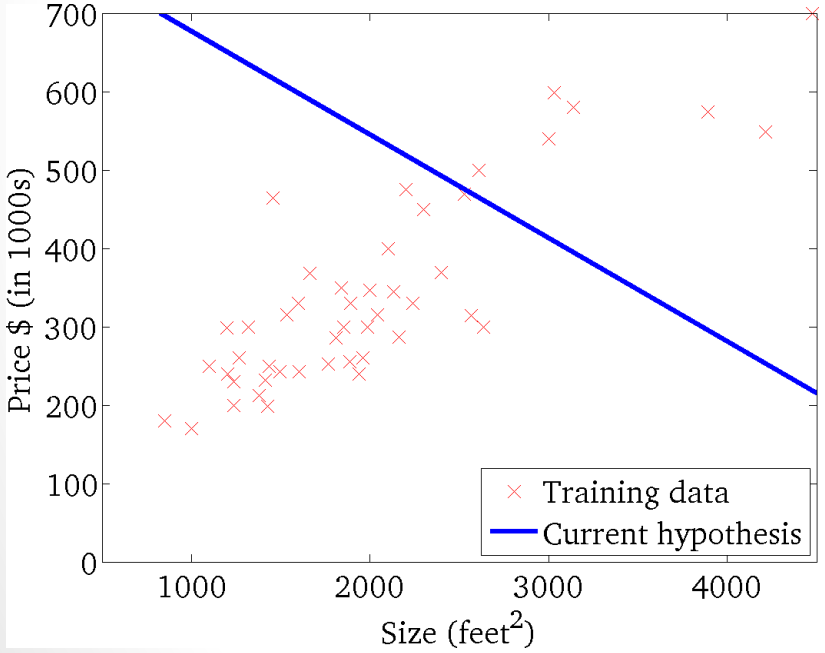
$\boldsymbol{J(\theta_0, \theta_1)}$

- (function of the parameters θ_0, θ_1)



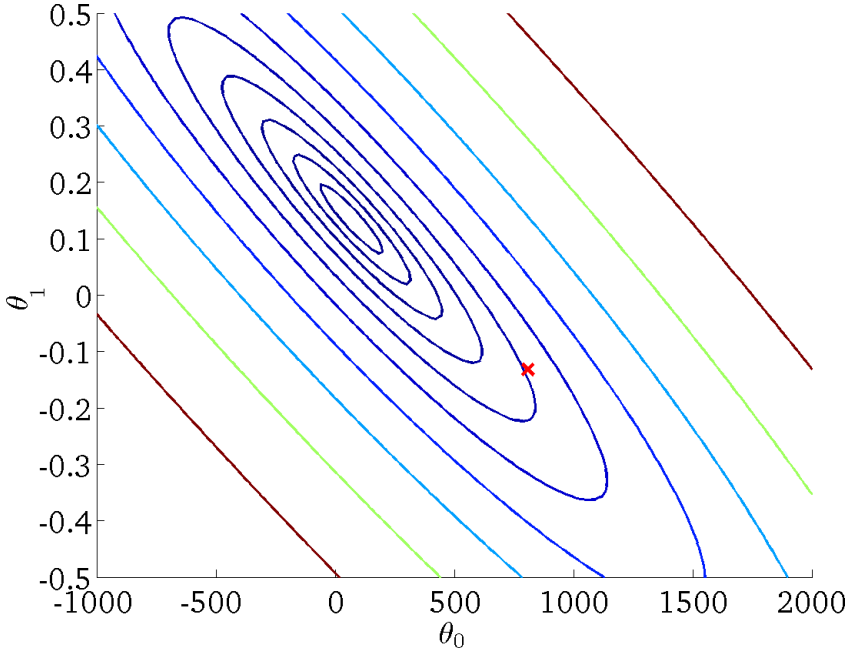
Using both "Knobs"

- for a fixed θ_0, θ_1 this is a function of x
  - $\boldsymbol{h\theta(x) = \theta_0 + \theta_1 x}$



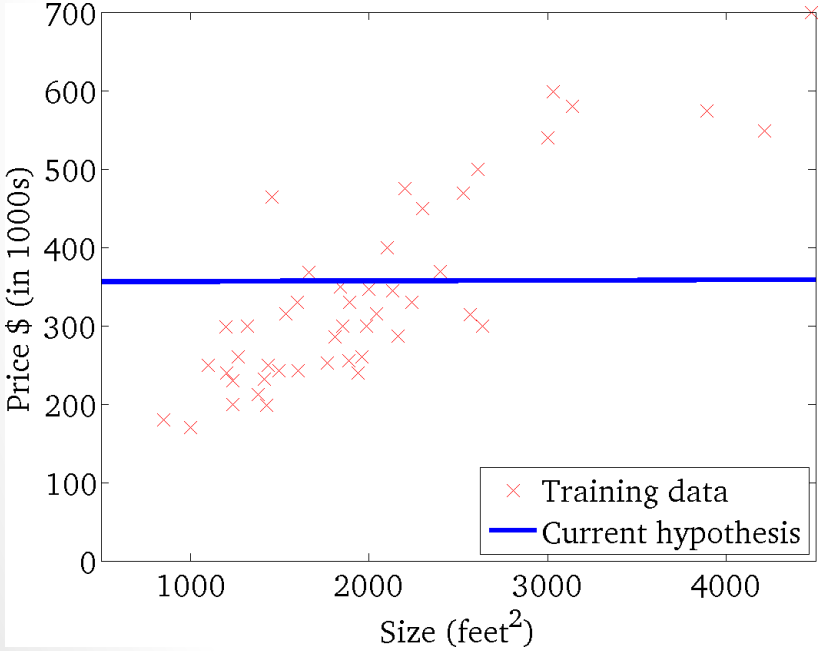
$$\boldsymbol{J(\theta_0, \theta_1)}$$

- (function of the parameters θ_0, θ_1)



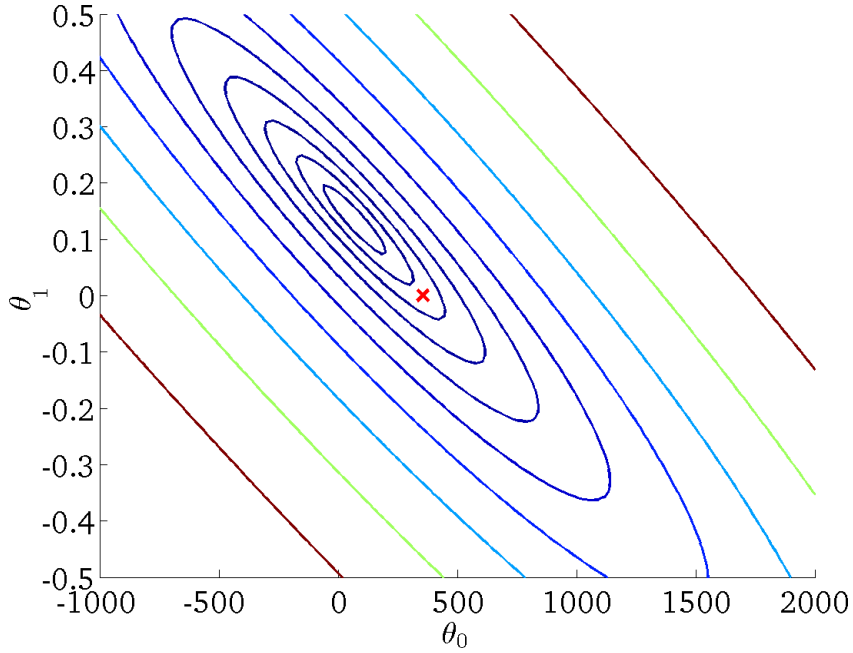
Using both "Knobs"

- for a fixed θ_0, θ_1 this is a function of x
  - $\boldsymbol{h\theta(x) = \theta_0 + \theta_1 x}$



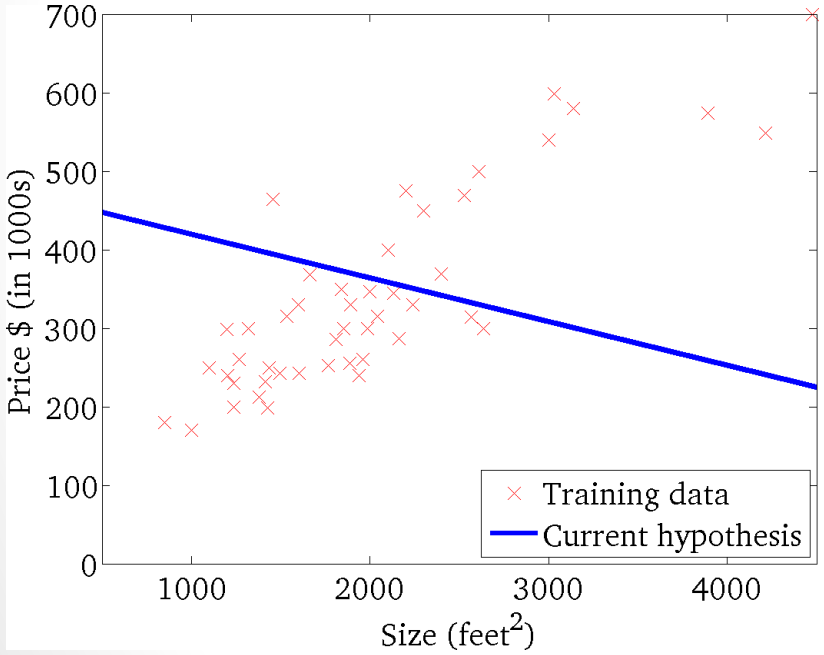
$$\boldsymbol{J(\theta_0, \theta_1)}$$

- (function of the parameters θ_0, θ_1)



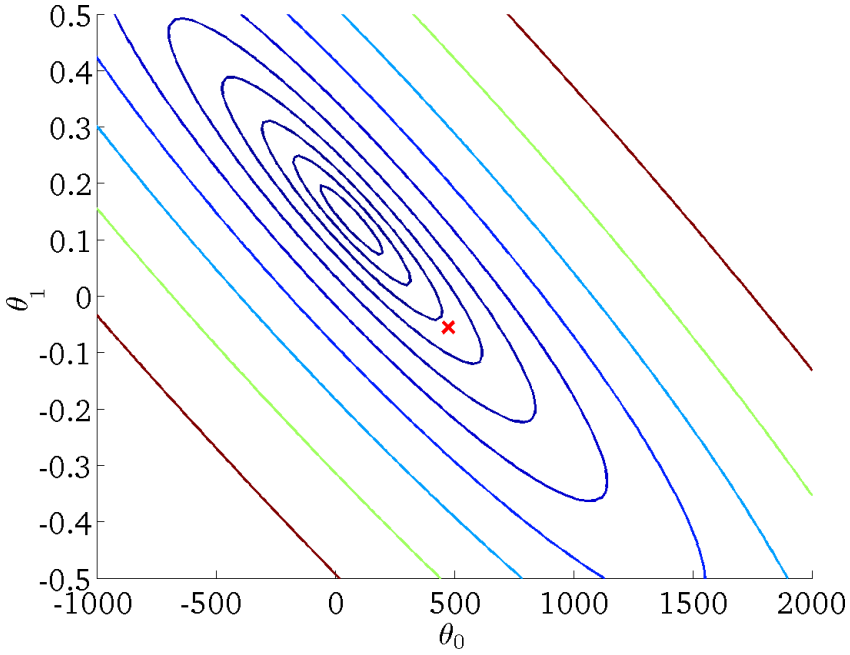
Using both "Knobs"

- for a fixed θ_0, θ_1 this is a function of x
  - $\boldsymbol{h\theta(x) = \theta_0 + \theta_1 x}$



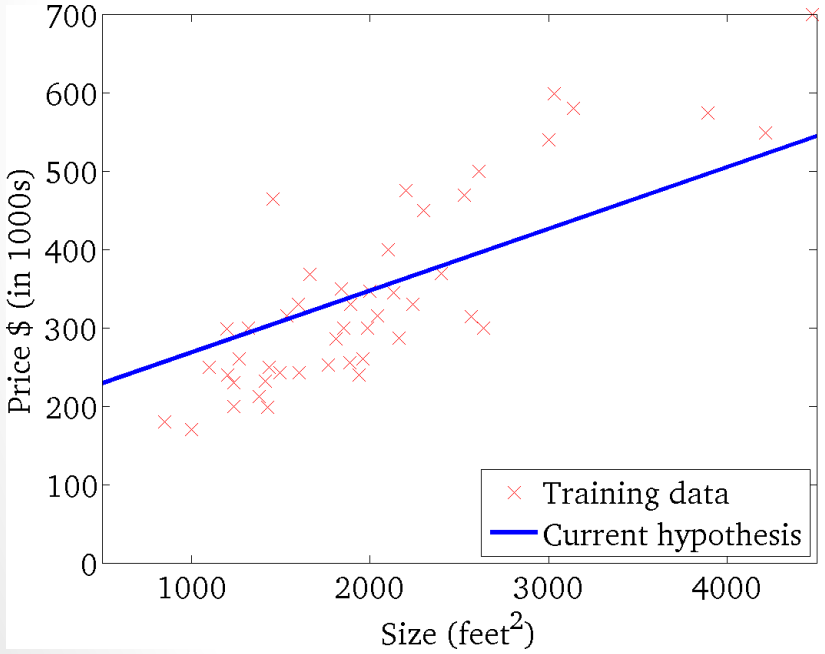
$$\boldsymbol{J(\theta_0, \theta_1)}$$

- (function of the parameters θ_0, θ_1)



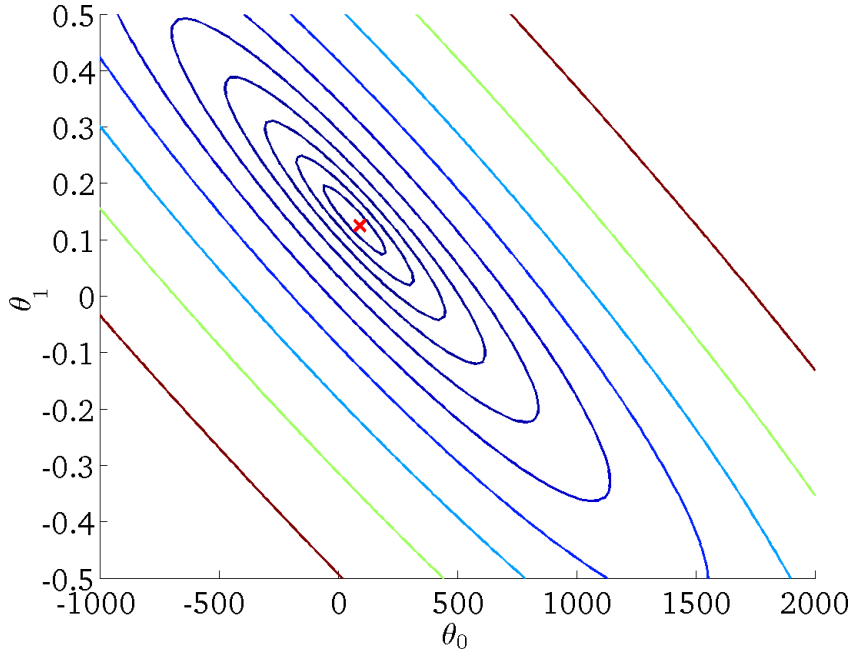
Using both "Knobs"

- for a fixed θ_0, θ_1 this is a function of x
  - $\boldsymbol{h\theta(x) = \theta_0 + \theta_1 x}$



$$\boldsymbol{J(\theta_0, \theta_1)}$$

- (function of the parameters θ_0, θ_1)

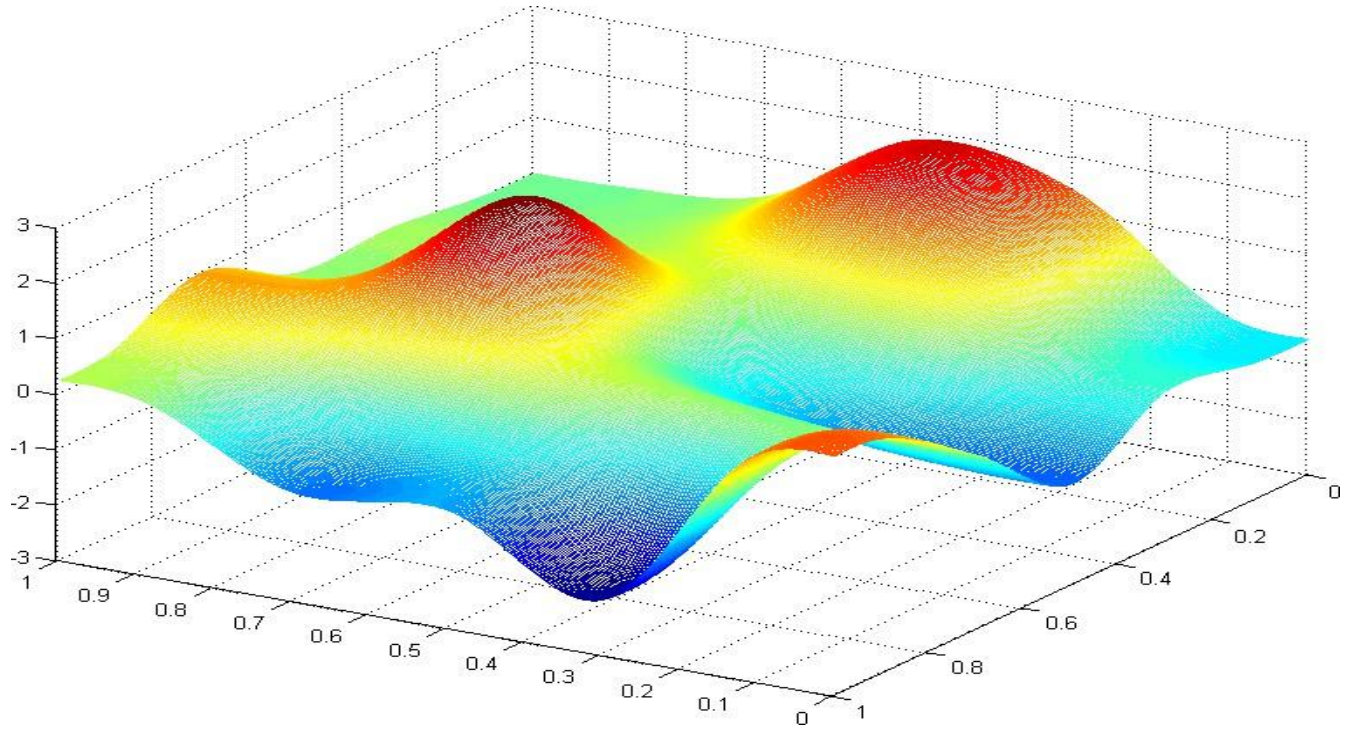


Gradient Descent

The Gradient Descent Algorithm

- **Goal:** Minimize $J(\theta_0, \theta_1)$
- **Outline:**
  - Start with some θ_0, θ_1 - For example (0,0)
  - Keep updating θ_0, θ_1 to reduce $J(\theta_0, \theta_1)$
    - Until we **<u>hopefully</u>** reach **<u>a</u>** minimum

Gradient Descent



A simplified version of gradient descent

- Assume again that we set $\theta 0 = 0$ and our hypothesis and cost function practically have only one coefficient,$\theta 1$.

$$\boldsymbol{h\theta(x) = \theta_1 x}$$

- $repeat\ until\ convergence\{$

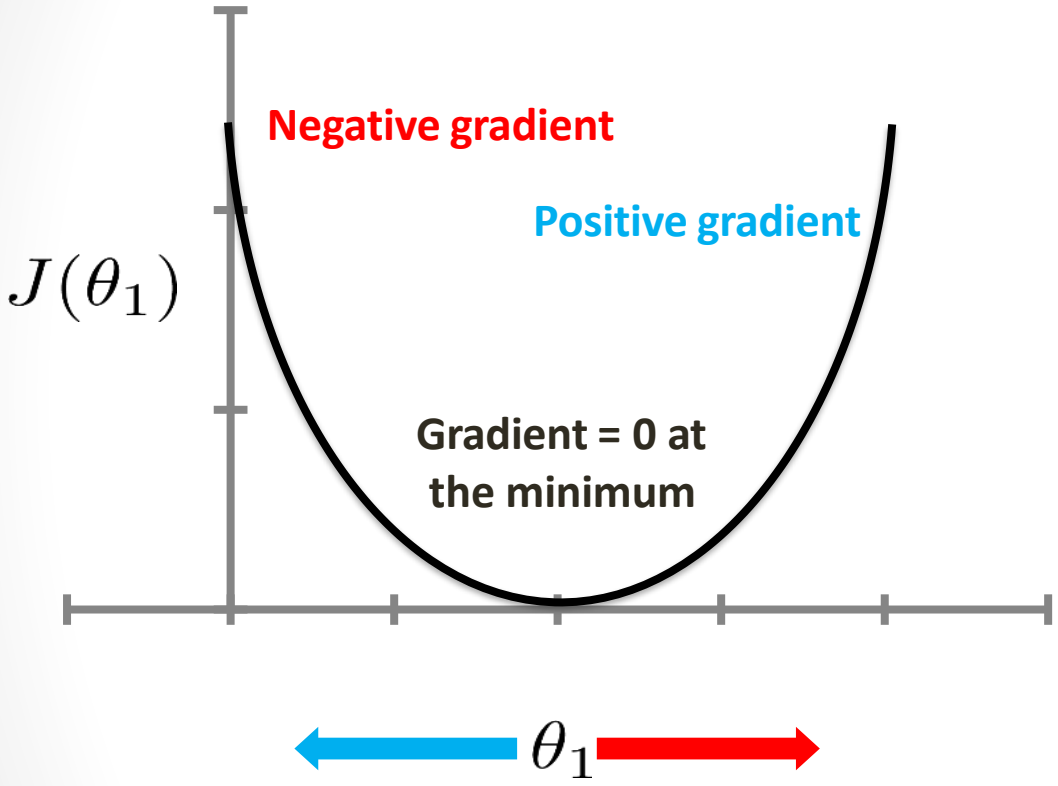
$$\theta_1 := \theta_1 - \frac{d}{d(\theta_1)}\left(J(\theta_1)\right)$$

}

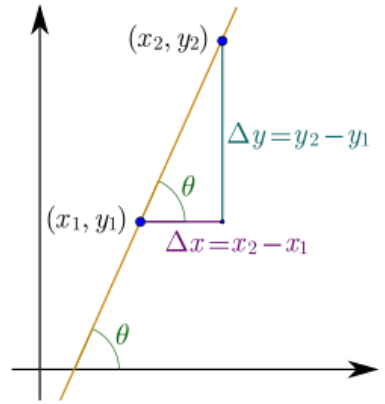
- Where

$$\boldsymbol{J(\theta_0, \theta_1) = \frac{1}{2m}\sum_{i=1}^{m}(h\theta(xi) - yi)^2}$$

Direction of step



$$\theta_1 := \theta_1 - \frac{\partial}{\partial(\theta_1)} (J(\theta_1))$$



Gradient = $\frac{y_2 - y_1}{x_2 - x_1} = \frac{f(x_2) - f(x_1)}{x_2 - x_1}$

The limiting case is when x_2 approaches x_1

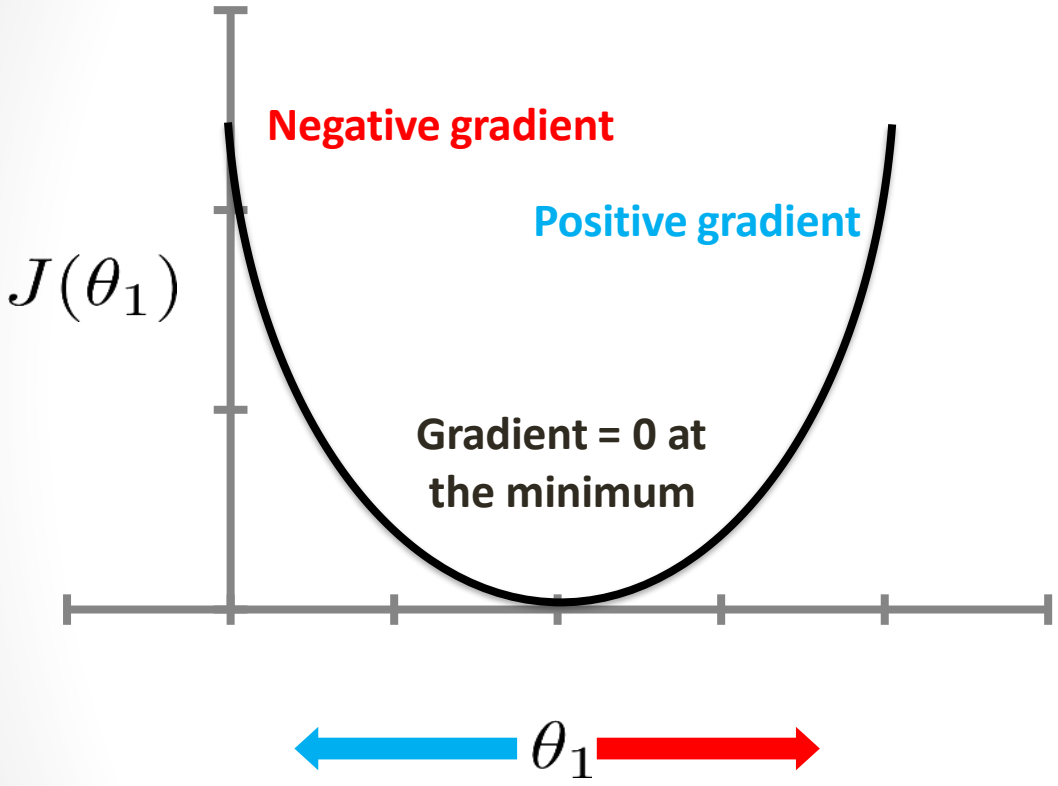
$= \frac{\Delta y}{\Delta x} as \Delta x \rightarrow 0 = \frac{dy}{dx}$

$$\theta_1 := \theta_1 - \alpha \frac{d}{d\theta_1} (J(\theta_1))$$

$\theta_1 := \theta_1 - \alpha(negative)$: Increases θ_1

$\theta_1 := \theta_1 - \alpha(positive)$: Decreases θ_1

Step Size



$$\theta_1 := \theta_1 - \frac{\partial}{\partial(\theta_1)} (J(\theta_1))$$

Gradient = $\frac{y_2 - y_1}{x_2 - x_1} = \frac{f(x_2) - f(x_1)}{x_2 - x_1}$

The limiting case is when x_2 approaches x_1 $= \frac{\Delta y}{\Delta x} as \Delta x \rightarrow 0 = \frac{dy}{dx}$

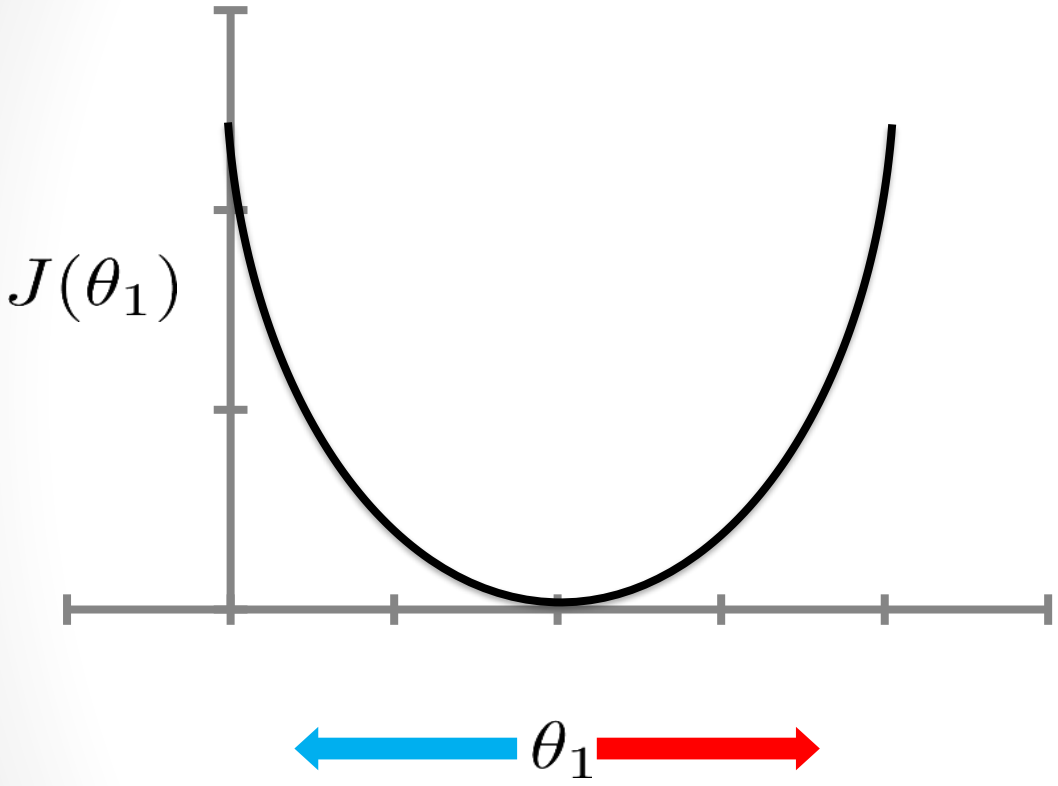
$$\theta_1 := \theta_1 - \alpha \frac{d}{d\theta_1} (J(\theta_1))$$

$\theta_1 := \theta_1 - \alpha(negative)$: Increases θ_1

$\theta_1 := \theta_1 - \alpha(positive)$: Decreases θ_1

- In addition to α, the steepness of the gradient also control the step size.
- The step sizes become smaller as we get closer to the minimum, even with a fixed α.
- With α too small, takes a long time to reach the minimum

Step Size



$$\theta_1 := \theta_1 - \frac{\partial}{\partial(\theta_1)} (J(\theta_1))$$

Gradient = $\frac{y_2 - y_1}{x_2 - x_1} = \frac{f(x_2) - f(x_1)}{x_2 - x_1}$

The limiting case is when x_2 approaches x_1 $= \frac{\Delta y}{\Delta x}$ $as \Delta x \rightarrow 0 = \frac{dy}{dx}$

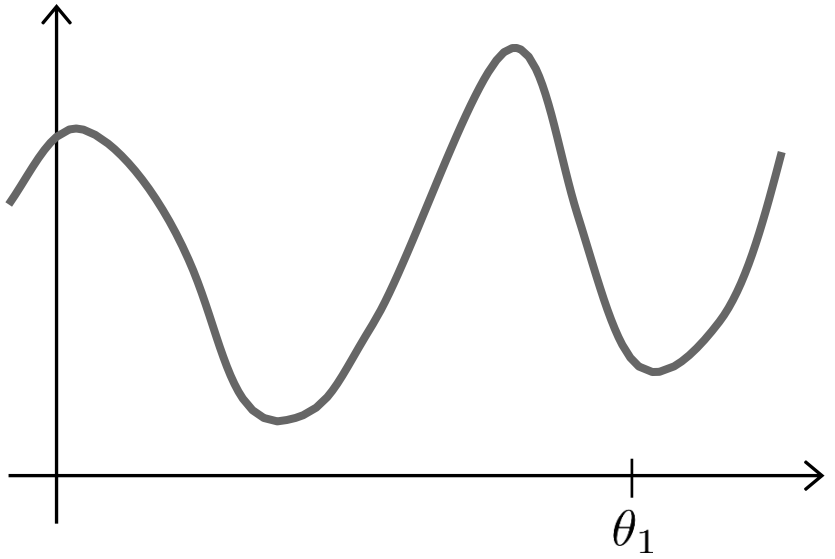
$$\theta_1 := \theta_1 - \alpha \frac{d}{d\theta_1} (J(\theta_1))$$

$\theta_1 := \theta_1 - \alpha(negative)$: Increases θ_1

$\theta_1 := \theta_1 - \alpha(positive)$: Decreases θ_1

- In addition to α, the steepness of the gradient also control the step size.
- The step sizes become smaller as we get closer to the minimum, even with a fixed α.
- With α too small, takes a long time to reach the minimum
- With α too big, we can miss the minimum, and may fail to converge

The problem of local optima



Gradient descent algorithm

- *repeat until convergence*{

$$\theta_j := \theta_j - \frac{\partial}{\partial(\theta_1)} J(\theta_0, \theta_1)) \qquad \text{(for j = 0 and j = 1)}$$

}

- Where α is the learning rate. E.g. 1, 0.1, 0.01, 0.001,… etc.

- **Correct:** Simultaneous Update

$$\text{temp0} := \theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1)$$
$$\text{temp1} := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$$
$$\theta_0 := \text{temp0}$$
$$\theta_1 := \text{temp1}$$

- **Incorrect**

$$\text{temp0} := \theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1)$$
$$\theta_0 := \text{temp0}$$
$$\text{temp1} := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$$
$$\theta_1 := \text{temp1}$$

Gradient descent algorithm

- $repeat\ until\ convergence\{$

$$\theta_j := \theta_j - \frac{\partial}{\partial(\theta_1)} J(\theta_0, \theta_1)) \qquad (\text{for j} = 0 \text{ and j} = 1)$$

}

- Where α is the learning rate. E.g. 1, 0.1, 0.01, 0.001,… etc.

$$\boldsymbol{J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} (h\theta(xi) - yi)^2}$$

Gradient Descent Algorithm

- $repeat\ until\ convergence\{$

$$\theta_j := \theta_j - \frac{\partial}{\partial(\theta_1)} J(\theta_0, \theta_1))$$

$$(\text{for j} = 0 \text{ and j} = 1)$$

$\}$

Linear Regression

$$\boldsymbol{h_\theta(x) = \theta_0 + \theta_1 x}$$

$$\boldsymbol{J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} (h\theta(xi) - yi)^2}$$

Gradient descents

- Batch Gradient Descent:
  - Batch gradient descent (SGD) calculates the gradient using a the entire dataset in each iteration.
  - therefore, requires the entire training dataset to be in memory and available to the algorithm.
- Stochastic Gradient Descent.
  - Stochastic gradient descent (SGD) calculates the gradient using a single sample at a time.
  - In contrast to Batch GD, Stochastic GD updates the model parameters in each epoch by considering on record at a time. The advantage is it may converge faster and is not memory hungry like Batch GD.
- Mini Batch
  - Suppose you have 100 observations you make four batches of 25 observations each and in turn you get four betas. Finally you select the best betas for your model.
  - Mini-batch gradient descent is a combination of the concepts of SGD and batch gradient descent. It simply splits the training dataset into small batches and performs an update for each of those batches. This creates a balance between the robustness of stochastic gradient descent and the efficiency of batch gradient descent.

Thank You